Phish Scan

Email Credibility & Phishing Detection Tool

IT 357 | Network Security | Group Project Report

Author: James Yahr, Bosun Moibi, Denvour Davis, Christopher Vandergriff

1. Introduction & Purpose

Phishing remains one of the most prevalent and effective attack vectors in modern cybersecurity. According to the Anti-Phishing Working Group (APWG), phishing attacks reached record highs in recent years, with hundreds of thousands of unique campaigns observed per quarter. Despite advances in enterprise email filtering, individual users continue to be targeted and deceived by increasingly convincing fraudulent messages. The problem is not simply technical; it is educational. Many users do not know what to look for, and even technical indicators such as SPF failures or Reply-To mismatches are invisible to recipients using standard email clients.

We created this tool to help solve that issue. The tool is a browser-based email threat analyzer that accepts a standard .eml file, performs automated static analysis against a set of known phishing indicators, generates a numeric risk score, and then leverages a locally running large language model (LLM) via LM Studio to produce a plain-English explanation of the findings. The goal is twofold: detection and education. Not only does the tool identify whether an email is likely malicious; it explains why, giving users the context to recognize similar attacks in the future.

The rise of AI-generated content has made phishing even more difficult to detect by eye alone. Attackers now routinely use large language models to craft grammatically flawless, contextually convincing emails that lack the spelling errors and awkward phrasing traditionally used as warning signs. This shift makes automated, indicator-based analysis tools more relevant than ever. Human intuition alone is no longer a reliable defense. Phish Scan was built with this reality in mind, combining deterministic header and content analysis with AI-assisted explanation to provide a detection layer that does not rely on visual grammar cues.

Project Scope

Phish Scan is scoped as a client-side forensic analysis tool. It is not a real-time email filter or mail transfer agent plugin. Instead, it operates on exported .eml files, making it suitable for post-receipt analysis, security awareness training, and incident triage. The tool runs entirely in a web browser, with no server-side backend required. The only external dependency is an optional locally hosted LLM via LM Studio for the AI explanation feature.

Target Users

The intended audience includes IT security students and practitioners performing email forensics, help desk personnel triaging reported phishing emails, and end users who want to understand suspicious messages they have received. The tool is designed to be approachable for non-experts while remaining technically substantive for those with a security background. Because all analysis and output is displayed in plain language with severity labels and natural-language explanations, users without a deep technical background can still interpret results meaningfully and take appropriate action.

2. Technical Implementation

Phish Scan is implemented as a single self-contained HTML file with embedded CSS and JavaScript. This architecture was chosen deliberately: it requires no installation, no build toolchain, and no runtime dependencies beyond a modern web browser. The entire application (parser, analysis engine, scoring logic, and UI) is contained in one file of approximately 800 lines. The tool must be served over HTTP rather than opened directly from the filesystem, as browsers restrict fetch() API calls from file:// origins. A simple local server such as Python's built-in HTTP server or VS Code Live Server satisfies this requirement.

2.1 EML Parser

The parser processes raw RFC 2822 MIME-formatted email files entirely in JavaScript. It reads the file using the FileReader API and splits the content online breaks to separate the header section from the body. Header parsing handles folded headers (continuation lines beginning with whitespace) as specified in RFC 2822. The body is decoded based on the Content-Transfer-Encoding header, with support for both quoted printable and base64 encoding. Subject lines encoded in RFC 2047 encoded-word format (commonly used for non-ASCII characters) are also decoded. Authentication results (SPF, DKIM, DMARC) are extracted from the Authentication-Results header using regular expressions. All URLs present in the raw file, including those in both headers and body, are extracted into a deduplicated array.

2.2 Static Analysis Engine

The analysis engine evaluates the parsed email against twelve distinct checks, organized into three scoring categories: header analysis, content analysis, and link analysis. Each check contributes a weighted point value to its respective category score. The three category scores are summed to produce a total risk score capped at 100. Score thresholds map to three verdict levels:

- 0 – 34: CLEAN: no significant phishing indicators detected
- 35 – 69: SUSPICIOUS: ambiguous indicators warrant caution
- 70 – 100: PHISHING: multiple high-confidence indicators present

The following table summarizes each check and its scoring contribution:

Indicator	Score Contribution
Reply-To domain mismatch	+30 (header)
Return-Path domain mismatch	+25 (header)
Brand impersonation in display name	+30 (header)
SPF fail/softfail (from .eml header only)	+8 (header)
DKIM fail/none (from .eml header only)	+6 (header)
DMARC fail/none (from .eml header only)	+4 (header)
Missing Message-ID header	+10 (header)

Indicator	Score Contribution
Urgency/manipulation keywords in subject	+20 (content)
Sensitive information request in body	+35 (content)
Threat/urgency language in body	+20 (content)
Suspicious URL pattern (per link, first match)	+15 (links)
Excessive link count (>8 links)	+10 (links)

2.3 Authentication Result Limitations

A deliberate design decision was made regarding SPF, DKIM, and DMARC checks. Phish Scan reads these values from the Authentication-Results header already present in the .eml file; these values were written by the receiving mail server at delivery time. The tool does not perform live DNS lookups to independently verify these results, as the browser environment does not support raw DNS queries. As a consequence, the point contributions for auth failures were intentionally reduced to +8, +6, and +4 respectively, and each indicator is displayed with an explicit caveat noting that no live verification was performed. This prevents the tool from returning inflated risk scores on emails where the Authentication-Results header may be absent or manually constructed.

2.4 AI Integration (LM Studio)

After static analysis completes, Phish Scan sends a structured prompt to a locally running LM Studio instance via its OpenAI-compatible REST API endpoint (default: http://localhost:1234/v1/chat/completions). The prompt includes the risk score, verdict, all detected indicators, extracted URLs, and the first 600 characters of the email body. Critically, the prompt instructs the model to explain and contextualize the static analysis verdict rather than perform its own independent assessment. This prevents the LLM from overriding a CLEAN verdict based on its general training data about phishing patterns. The system prompts frames the model as a neutral analyst whose role is explanation, not detection. Temperature is set to 0.3 for consistent, factual output.

The tool degrades gracefully when LM Studio is unavailable, as the static analysis results are always displayed regardless of AI connectivity, and the AI panel shows a clear error message with the raw server response for debugging.

3. Justification & Analysis

3.1 Whya Browser-Based Single-File Tool

The choice to implement Phish Scan as a single HTML file rather than a Python command-line script or a full web application with a backend server was driven by portability and accessibility. A Python script requires compatible runtime, installed dependencies, and command-line familiarity. A full web application requires server deployment. A single HTML file can be distributed, opened on any operating system with a browser, and used immediately, with the only setup step being to serve it over a local HTTP server to satisfy browser security restrictions. For a security awareness tool intended to reach non-technical users, this tradeoff strongly favors the browser-based approach.

3.2 Why Local LLM Rather Than a Cloud API

Integrating AI analysis via LM Studio rather than a cloud-based API such as OpenAI or Anthropic was a deliberate privacy decision. Email files may contain sensitive personal information, including names, financial details, and internal communications. Sending this content to a third-party cloud of API introduces data privacy risks and potential compliance concerns, particularly in enterprise or educational contexts. LM Studio runs entirely on the local machine, meaning no email content ever leaves the user's system. This design aligns with the principle of data minimization and makes the tool suitable for environments with strict data handling requirements.

3.3 Indicator Selection Rationale

The twelve indicators selected for the static analysis engine are grounded in established phishing detection research and threat intelligence reporting. Reply-To and Return-Path domain mismatches are among the most reliable header-based indicators of spoofed sender identity, as legitimate transactional email almost never routes replies or bounces to a different domain than the sender. Brand impersonation detection targets display name spoofing, a technique explicitly documented in MITRE ATT&CK under T1566.001 (Spear phishing Attachment) and T1598.003 (Phishing for Information via Service). Urgency language and sensitive information requests are characteristic of social engineering attacks documented in NIST SP 800-177r1 (Trustworthy Email). URL analysis covers shortener abuse, raw IP addressing, unencrypted HTTP links, and abused free TLDs, all consistently observed in phishing campaigns reported by CISA and documented in the APWG eCrime research corpus.

3.4 Scoring Model Limitations

The scoring model is intentionally simple and transparent. Each indicator contributes to to a fixed weight regardless of context, which means the tool can produce false positives on legitimate marketing emails that use urgency language, URL shorteners, or large link counts. The model does not account for sender's reputation, domain age, or WHOIS data, as these would require network lookups not feasible in a browser context. These limitations are acknowledged as acceptable tradeoffs for a v1 tool focused on education and triage rather than production-grade filtering. The AI analysis layer partially compensates for this by providing contextual nuance that the rule-based engine cannot, for example distinguishing between a URL shortener used in a newsletter versus one used in an isolated credential-harvesting email.

3.5 PromptEngineering Considerations

A significant challenge encountered during development was the tendency of the language model to override the static analysis verdict. Early versions of the prompt framed the model as a 'phishing investigator,' which caused it to identify threat patterns even in emails that scored CLEAN, producing a false positive rate that undermined user trust in the tool. The solution was to restructure the system promptly position the model as a neutral analyst whose role is to explain the existing score, not generate an independent one. Verdict-conditional instructions were added to the user prompt specifying exactly what the model should address for each of the three possible outcomes. Temperature was lowered to 0.3 to reduce hallucinated threat patterns. These changes produced substantially more accurate and consistent AI output across both phishing and legitimate email samples.

4. Conclusion

Phish Scan demonstrates that effective phishing detection tooling does not require a complex infrastructure stack. By combining a lightweight RFC 2822 parser, a rule-based static analysis engine,

and a locally hosted large language model, the tool delivers both a quantitative risk assessment and a qualitative explanation in a single browser session, with no data leaving the user's machine. The deliberate choice to keep all components local (the analysis engine, the scoring model, and the AI) means the tool can be deployed in privacy-sensitive environments where sending email content to external services is not acceptable.

The project surfaced with several meaningful technical lessons. The limitations of browser-based DNS resolution required a careful rethinking of how authentication results are weighted and communicated to the user. Naively assigning full weight to SPF, DKIM, and DMARC failures would have inflated scores on emails with missing or synthesized headers, undermining the credibility of the verdict. The solution, reduced weighting combined with an explicit on-screen caveat, reflects a broader principle in security tooling: it is better to be transparent about a limitation than to hide it behind a confident but unreliable result.

The challenge of preventing the LLM from overriding deterministic analysis results highlighted the importance of precise prompt engineering. Framing, role assignment, and verdict-conditional instructions all proved necessary to keep the model acting as an explainer rather than an independent classifier. Temperature tuning further reduced hallucinated threat patterns. These prompt engineering decisions are as much a part of the technical implementation as the parser or the scoring engine, and they represent a non-trivial portion of the overall development effort.

From an educational standpoint, Phish Scan achieves its core objective: it makes the technical mechanics of phishing attacks visible and understandable. A user who sees a HIGH indicator explaining that the display name says 'PayPal', but the actual sending domain is paypal-accounts-verification.com learns something concrete about how spoofing works, not just that an email is dangerous, but why. The combination of a numeric score, color-coded severity badges, and a plain-language AI explanation gives users at every technical level something actionable to take away from the analysis.

Future Work

Several enhancements would meaningfully strengthen the tool in future iterations. Integration with a DNS-over-HTTPS provider (such as Cloudflare's 1.1.1.1 API) would enable live SPF and DKIM verification from the browser, replacing the current dependency on pre-existing header values. A domain reputation lookup against a public threat intelligence feed (such as the SANS ISC or VirusTotal public API) would add sender context that the current scoring model lacks. Support for multi-part MIME parsing would improve body extraction accuracy for HTML emails with embedded images or attachments. A batch analysis mode allowing multiple .eml files to be queued and compared would be valuable for incident response use cases where analysts need to assess a campaign of related messages simultaneously.

Longer term, the fixed-weight scoring model could be supplemented with a machine learning classifier trained on labeled phishing datasets such as the APWG eCrime corpus. A trained classifier would handle contextual false positives (such as legitimate marketing emails that trigger urgency keyword rules) more gracefully than static rules. The local LLM integration already provides a degree of contextual reasoning, but a purpose-trained classification layer would produce more consistent and auditable verdicts. Packaging the tool as a browser extension that analyzes emails directly within a webmail interface would also substantially reduce the friction of the current export-and-load workflow, making Phish Scan accessible to a much broader audience.